

Interfaces de comunicación en sistemas embebidos

Comunicación serial: I2C

Juan José Londoño, M.Sc.

Introducción

En sistemas embebidos no existe un único protocolo de comunicación que sirva para todo, cada protocolo nace para resolver una necesidad específica.

UART, por ejemplo, es muy útil para comunicación serial simple entre **dos dispositivos**, pero cuando queremos conectar **varios periféricos** de manera ordenada y usando pocas líneas, aparecen otros protocolos más convenientes, como I2C.

Introducción

I2C significa ***Inter–Integrated Circuit***.

Es un protocolo de comunicación serial, síncrono, orientado a la comunicación entre circuitos integrados dentro de un mismo sistema electrónico.

I2C fue pensado para que un microcontrolador se comuniquen con otros chips en la misma PCB o en **distancias cortas**, como sensores, memorias EEPROM, convertidores ADC/DAC, etc.

Arquitectura de I2C

I2C usa dos líneas:

SDA (Serial Data Line):

Por esta línea viajan los bits de datos.

SCL (Serial Clock Line):

Por esta línea viaja la señal de **reloj** que sincroniza la transmisión.

Ambas líneas son compartidas por todos los dispositivos del bus.

Arquitectura de I2C

En I2C, todos los dispositivos conectados al bus comparten las dos líneas. Esto significa que tanto el microcontrolador como los periféricos están conectados en paralelo a SDA y SCL.

El reloj permite que todos los dispositivos sepan exactamente en qué momento debe interpretarse cada bit.

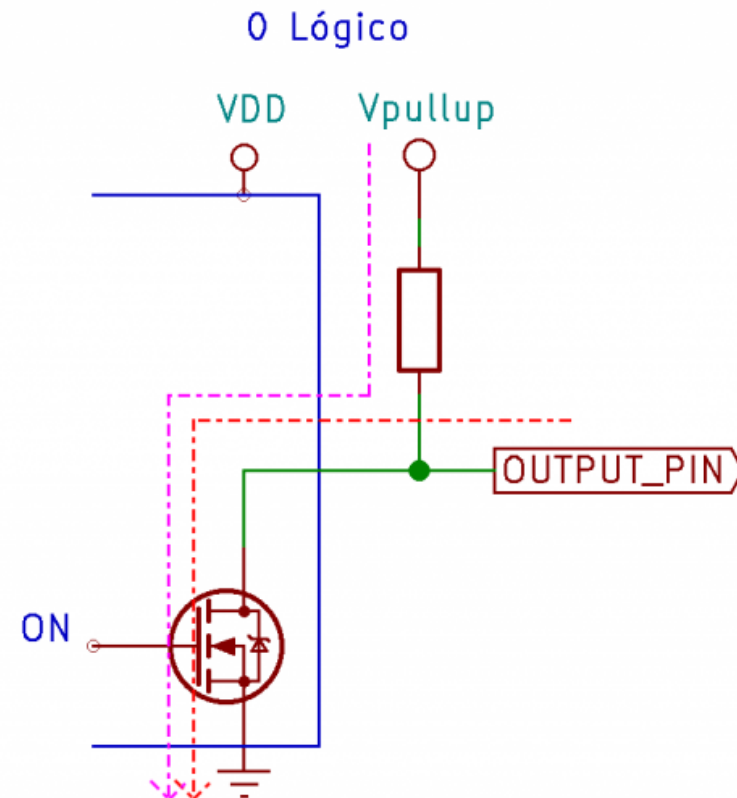
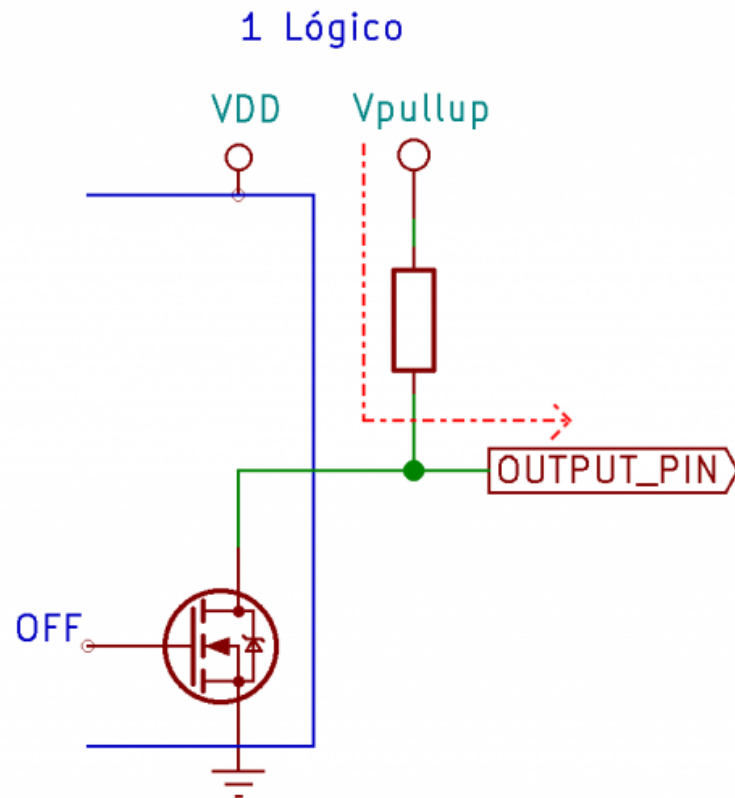
Arquitectura de I2C

I2C tiene una particularidad eléctrica muy importante:

Las líneas SDA y SCL no se manejan normalmente como salidas push–pull convencionales, sino como líneas de tipo open–drain, esto significa que los dispositivos pueden llevar la línea a nivel bajo, pero no la fuerzan activamente a nivel alto.

Para que la línea suba a nivel alto, se utilizan resistencias pull–up conectadas a la alimentación.

Líneas open drain



Roles en I2C

En I2C existen dos tipos de dispositivos:

Maestro (Master):

Es el dispositivo que inicia la comunicación, genera la señal de reloj, decide con qué dispositivo hablar y define si la operación será de lectura o de escritura.

Esclavo (Slave):

Es el dispositivo que permanece esperando una solicitud, responde únicamente cuando es direccionado y recibe datos o entrega datos según lo que el maestro solicite.

Roles en I2C

En la mayoría de las aplicaciones embebidas, el microcontrolador actúa como maestro y los periféricos como esclavos.

En I2C, la comunicación no es libre ni espontánea, el maestro organiza y controla toda la transacción.

Los términos clásicos de Maestro/Esclavo, hoy en día se suelen cambiar por **Controller/Target**.

Tipos de operación en I2C

El maestro puede realizar dos tipos principales de operación:

Escritura:

El maestro envía datos al esclavo.

Lectura:

El maestro solicita datos al esclavo.

Confirmación de recepción ACK y NACK

Después de ciertos bytes transmitidos en I2C, el receptor responde con un bit de confirmación.

ACK (Acknowledge)

Indica que el dato fue reconocido o aceptado.

NACK (Not Acknowledge)

Indica que no hubo reconocimiento o que no se desea continuar.

Confirmación de recepción ACK y NACK

Este mecanismo le permite al protocolo verificar si la comunicación está siendo atendida correctamente.

Por ejemplo, si el maestro envía una dirección y nadie responde, no hay confirmación válida y si un dispositivo responde con ACK, significa que reconoció la transacción.

ACK y NACK son fundamentales para saber si el bus realmente está respondiendo como se espera.

Direccionamiento en I2C

Como varios dispositivos comparten el mismo bus, el maestro necesita una forma de seleccionar con cuál quiere comunicarse, para ello, cada dispositivo esclavo posee una **dirección**.

El direccionamiento permite que muchos dispositivos estén físicamente conectados a las mismas líneas, pero que solo uno participe en una transacción concreta.

Las direcciones de 7 bits son el formato más común en sistemas sencillos y periféricos comerciales.

Direccionamiento en I2C

El flujo del direccionamiento es el siguiente:

1. El maestro inicia la transacción.
2. El maestro envía la dirección del esclavo.
3. Solo el esclavo que reconoce esa dirección responde.
4. Los demás dispositivos ignoran la transacción.

Transacción I2C

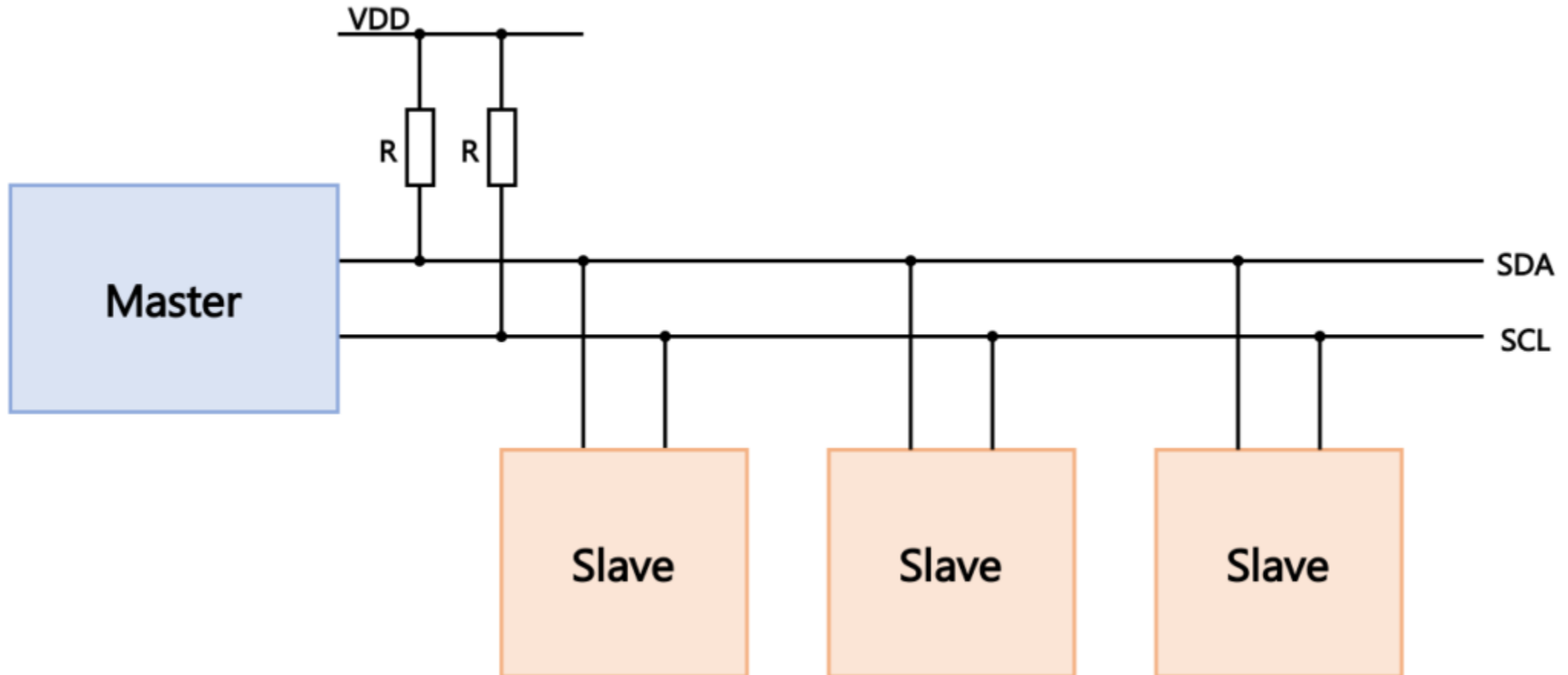
I2C no consiste simplemente en “mandar bytes”, toda transacción tiene una estructura organizada, donde primero debe identificarse el dispositivo destino y luego establecerse el tipo de operación.

Esto hace que el protocolo sea robusto y ordenado para sistemas con múltiples periféricos.

Transacción I2C

La secuencia de cada transacción tiene el siguiente esquema:

1. Inicio de la comunicación.
2. Envío de la dirección del esclavo.
3. Indicación de lectura o escritura.
4. Confirmación del esclavo.
5. Intercambio de uno o más bytes.
6. Finalización de la comunicación.



Velocidad del bus I2C

En I2C, la velocidad se asocia principalmente con la frecuencia del reloj del bus.

Existen 2 modos comunes:

Standard Mode: 100 kHz.

Fast Mode: 400 kHz.

Velocidad del bus I2C

Según el modo, se seleccionan ciertos valores de las resistencias de pull-up.

Standard Mode (100 kHz)

- valor típico usado: 4.7 k Ω
- rango común: 4.7 k Ω a 10 k Ω

Fast Mode (400 kHz)

- valor típico usado: 2.2 k Ω
- rango práctico común: 2.2 k Ω a 4.7 k Ω

I2C en el ESP32

El ESP32 incorpora controladores hardware dedicados para I2C, lo que permite implementar este protocolo sin necesidad de hacerlo completamente por software.

Gracias al hardware dedicado, el microcontrolador puede manejar las señales SDA y SCL, la generación del reloj y la sincronización de datos de manera automática.

I2C en el ESP32

El ESP32 posee dos controladores I2C hardware:

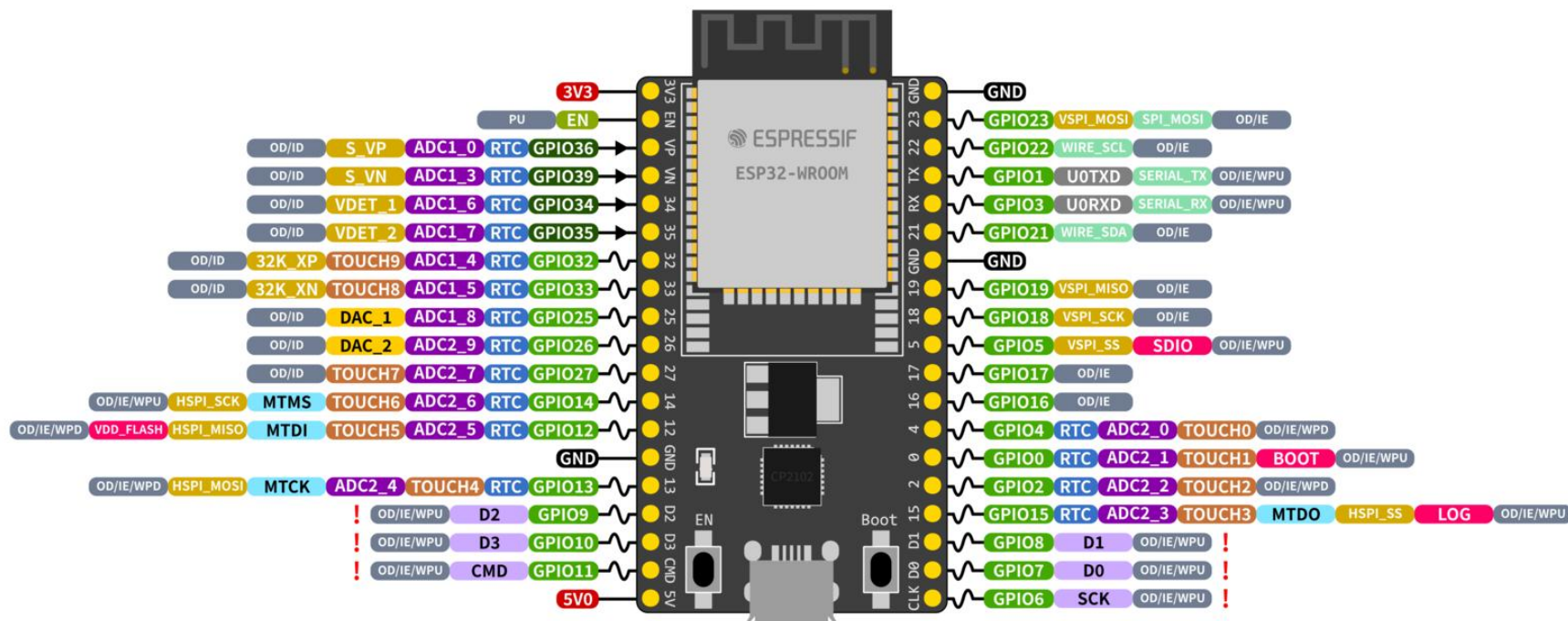
- I2C0
- I2C1

Cada uno puede configurarse de forma independiente en modo maestro o esclavo y además permite una configuración flexible de pines, por lo que permiten asignar las señales I2C a casi cualquier GPIO.

I2C en el ESP32

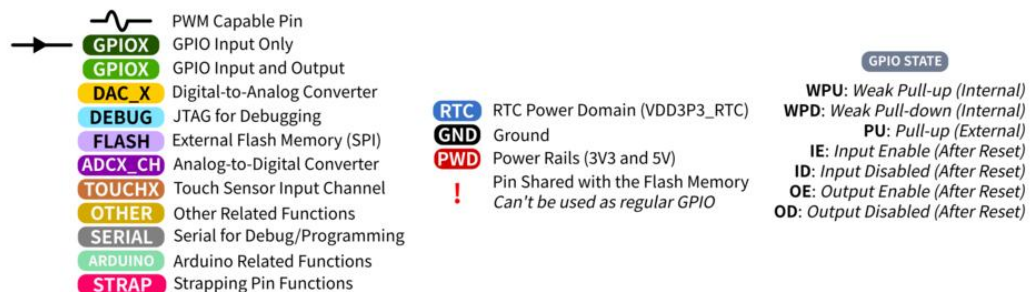
Generalmente los sistemas de desarrollo basados en ESP32, **NO incluyen** las **resistencias pull-up** de I2C, por lo que deben conectarse según la velocidad del bus, en caso de que el periférico no las incluya.





ESP32 Specs

32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz
 Bluetooth 4.2 BR/EDR and BLE
 520 KB SRAM (16 KB for cache)
 448 KB ROM
 34 GPIOs, 4x SPI, 3x UART, 2x I2C,
 2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
 1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet



Configuración de I2C



```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/i2c.h"
```


Configuración de I2C

```
#define I2C_PORT      I2C_NUM_0
#define I2C_SDA_IO    32
#define I2C_SCL_IO    33
#define I2C_FREQ_HZ   100000

#define SLAVE_ADDR    0x68
```

Configuración de I2C como maestro

```
i2c_config_t conf = {  
    .mode = I2C_MODE_MASTER,  
    .sda_io_num = I2C_SDA_IO,  
    .scl_io_num = I2C_SCL_IO,  
    .sda_pullup_en = GPIO_PULLUP_DISABLE,  
    .scl_pullup_en = GPIO_PULLUP_DISABLE,  
    .master.clk_speed = I2C_FREQ_H  
};  
  
i2c_param_config(I2C_PORT, &conf);  
i2c_driver_install(I2C_PORT, conf.mode, 0, 0, 0);
```

Construcción de una transacción I2C

```
i2c_cmd_handle_t cmd = i2c_cmd_link_create();  
i2c_master_start(cmd);  
i2c_master_write_byte(cmd, dato, true);  
i2c_master_read(cmd, buffer, len, I2C_MASTER_ACK);  
i2c_master_read_byte(cmd, &dato, I2C_MASTER_NACK);  
i2c_master_stop(cmd);
```

Ejecutar una transacción I2C



```
i2c_master_cmd_begin(I2C_PORT, cmd, pdMS_TO_TICKS(1000));  
i2c_cmd_link_delete(cmd);
```